

## 0.31.2. Deux lettres à la fois — doublé, deux positions

Quatre lettres, EN MÊME TEMPS : A et B à gauche (le 0.31), C et D en colonne 10.

### 1. Ce qu'il faut comprendre

C'est le 0.31 et le 0.31.1 réunis : les deux paires dans la même boucle.

Quatre `un_pas` dans la même image : les quatre lettres avancent ensemble. `ALPHABET[2]` est le C, `ALPHABET[3]` le D.

Le A s'arrête en colonne 9 (`if (xa < 9)`) : sans cela, il rattraperait la place du C.

#### Les fonctions de cette leçon — et le tuto de chacune

- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <un_pas>` → [0.29.4. La fonction un\\_pas\(\) — un seul pas, sans attendre](#)

### 2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1233 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage      87 o ← la console, à l'allumage — la mise en route de la console
un_pas()       117 o ← main() — une fonction fournie par le langage
main()         207 o ← démarrage — ta fonction
VBlank         8 o ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage  28 o ← image() — attend l'image suivante
AttendreVBlank 16 o ← démarrage — attend que l'écran soit libre
AttendreAcces  12 o ← EffacerCases — attend de pouvoir écrire une case
AdresseCase    10 o ← effacer(), poser() — trouve l'adresse d'une case de l'écran
EffacerCases    12 o ← effacer() — efface des cases
CopierTuiles   18 o ← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte   14 o ← démarrage — vide les deux cartes de l'écran
Tuiles         704 o ← CopierTuiles — la police entière
```

Lettres gravées : toutes (`poser()` reçoit un numéro calculé)

Pas gravé (jamais appelé) : `AvancerAirs`, `AttendreFinVBlank`, `EcrireTexte`, `EcrireNombre`, `Diviser`, `Reste`, `Multiplier`, `DecalerGauche`, `DecalerDroite`, `RangerLutins`, `InstallerTransfert`, `Hasard`, `EffacerFond`, `EffacerPanneau`, `LireManette`, `DonneesTransfert`, `Notes`

Fonctions du langage appelées :

```
poser() × 5    65 o sur place, dans un_pas(), main() + AdresseCase (10 o)
effacer() × 1   15 o sur place, dans un_pas() + AdresseCase (10 o), EffacerCases (12 o)
image() × 1     3 o sur place, dans main() + AttendreImage (28 o)
```

Tes fonctions appelées :

```
un_pas() × 4    88 o sur place, dans main()
```

## 1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
#include <poser>      // pose une tuile sur une case du fond
#include <ALPHABET>    // les lettres de la police : ALPHABET[0] est le A
#include <un_pas>      // fait faire un seul pas à une tuile
```

## 2. UNE VARIABLE — XA

```
uint8_t xa = 0;      // la colonne du A
```

## 3. UNE VARIABLE — YA

```
uint8_t ya = 0;      // sa ligne
```

## 4. UNE VARIABLE — XB

```
uint8_t xb = 0;      // la colonne du B
```

## 5. UNE VARIABLE — YB

```
uint8_t yb = 2;      // sa ligne
```

## 6. UNE VARIABLE — XC

```
uint8_t xc = 10;     // la colonne du C
```

## 7. UNE VARIABLE — YC

```
uint8_t yc = 0;      // sa ligne
```

## 8. UNE VARIABLE — XD

```
uint8_t xd = 10;     // la colonne du D
```

## 9. UNE VARIABLE — YD

```
uint8_t yd = 2;      // sa ligne
```

## 10. UNE VARIABLE — IMAGES

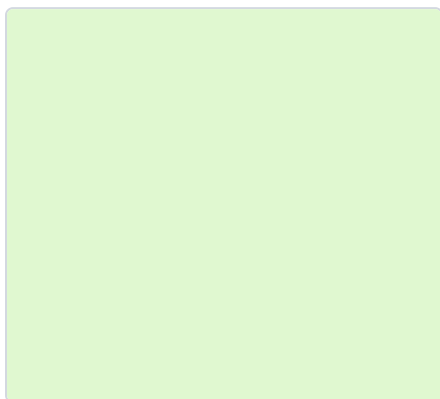
```
uint8_t images = 0;   // le chronomètre
```

## 11. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

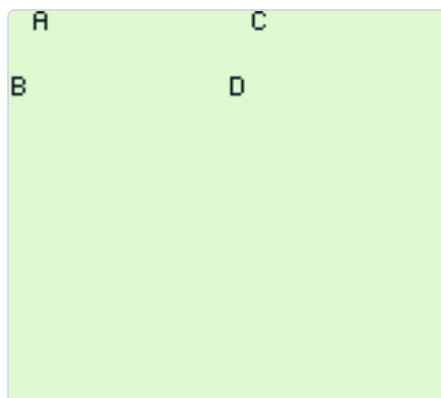
```
int main() {
    poser(xa, ya, ALPHABET[0]);
    poser(xb, yb, ALPHABET[1]);
    poser(xc, yc, ALPHABET[2]);
    poser(xd, yd, ALPHABET[3]);
    while (true) {
        image();
        images++;
        if (images == 15) {
            images = 0;
            if (xa < 9) un_pas(xa, ya, ALPHABET[0], 1, 0);
            if (yb < 16) un_pas(xb, yb, ALPHABET[1], 0, 1);
            un_pas(xc, yc, ALPHABET[2], 1, 0);
            if (yd < 16) un_pas(xd, yd, ALPHABET[3], 0, 1);
        }
    }
}
```

### 3. Ce qui se passe, étape par étape

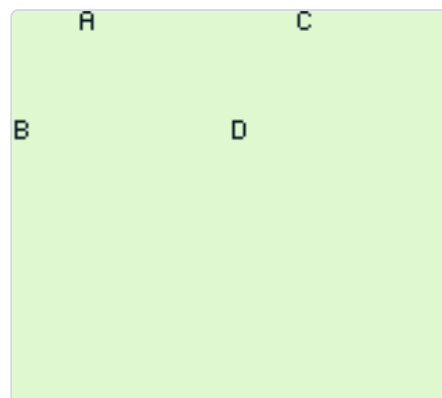
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



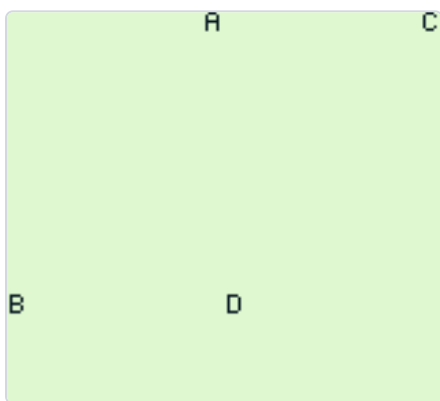
la toute première image — la console vient de démarrer — image 1



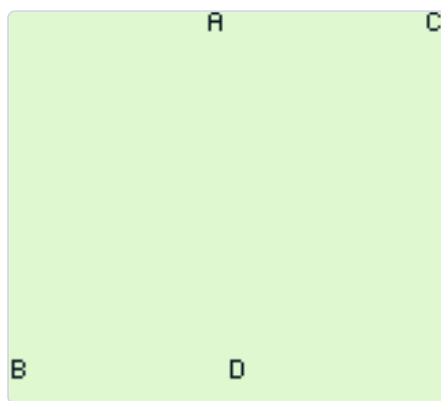
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



au bout de trois secondes — image 180



au bout de sept secondes — image 420

### 4. Ce qu'on doit voir

Quatre lettres en même temps : deux vont à droite sur la ligne du haut, deux descendent.

### 5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ le A s'arrête en 9 ( $x_a = 9$ )
- ✓ le C va jusqu'au bord ( $x_c = 19$ )
- ✓ le B descend jusqu'en 16 ( $y_b = 16$ )
- ✓ le D aussi ( $y_d = 16$ )

Ensuite : **0.32. Le trajet dans un tableau** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++